



Spring MVC

En mode JSP / Servlet

1

Spring MVC JSP / Servlet

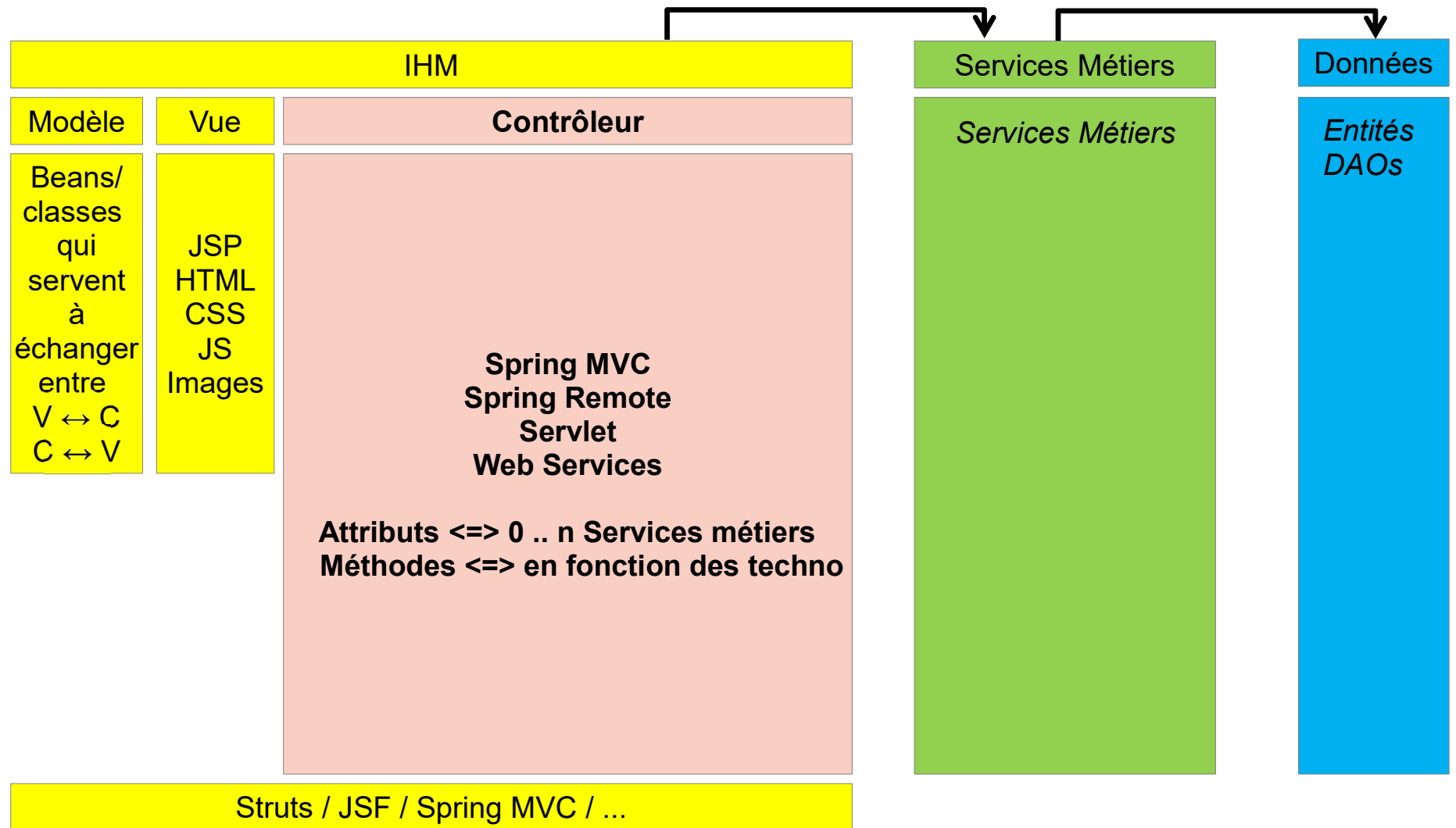
2

- ✓ Présentation
- ✓ Objets principaux
- ✓ Gestion des erreurs
- ✓ Gestion des validations de bean
- ✓ Internationalisation
- ✓ Thymeleaf

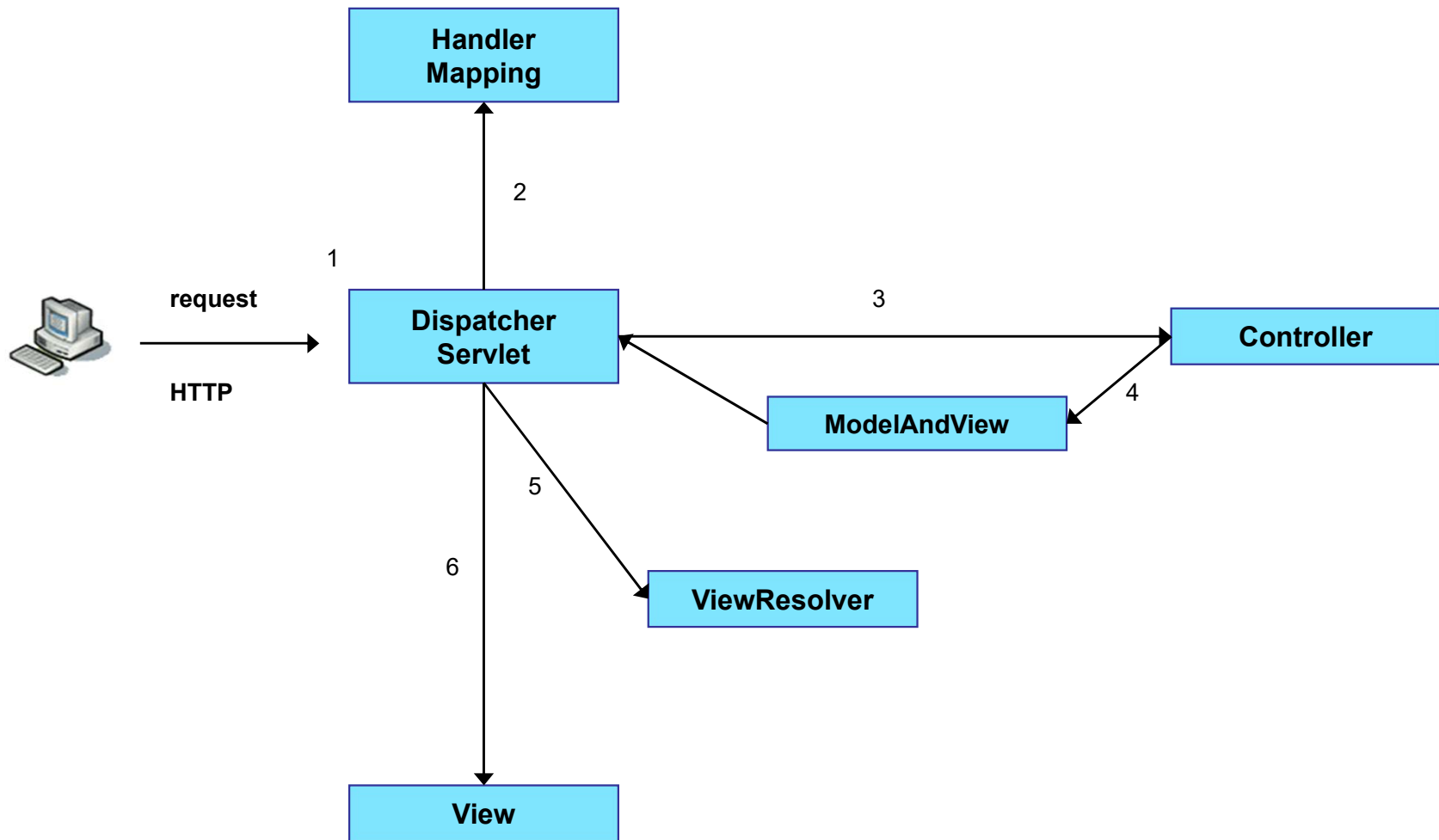
Présentation

- Ce chapitre présente uniquement la partie JSP/Servlet
 - Il ne contient pas la partie sur les services REST

Présentation



Cycle de vie d'une requête



Framework Spring MVC

- Il existe plusieurs manières de faire du Spring MVC
 - A l'ancienne ⇔ Struts 1 : héritage, méthode figée
 - ❑ On sait quoi écrire, on est comme dans une servlet
 - ❑ Simple, sans prise de tête si on vient du MVC standard JSP/Servlet (2000/2004)
 - ❑ On fait usage de `HttpServletRequest`, `HttpServletResponse`
 - Compiqué pour les tests unitaires ⇔ obligé de faire des tests d'intégrations sur la partie des contrôleurs
 - Moderne : annotation, méthodes dynamiques
 - ❑ On peut faire ce que l'on veut, paramètre, nom de méthodes, ...
 - ❑ On n'est plus obligé de faire usage de `HttpServletResponse`, `HttpSession`, `HttpServletRequest` ...
 - On peut faire des tests unitaires sur nos contrôleurs
 - ❑ Peut devenir complexe à gérer dans le temps, car chacun peut faire à sa sauce

Framework Spring MVC – Ancienne

➤ Vos contrôleurs

- Implémentent l'interface Controller
- Le contrôleur le plus simple proche de la servlet
- A vous de faire les `request.getParameter ...`

➤ Ou, si vous avez besoin d'un peu plus de service, vos contrôleurs peuvent hériter de `AbstractCommandController`

- Ce type de contrôleur permet un binding automatique des paramètres d'une requête vers un objet
 - ❑ Ex : sélection d'un élément dans une liste
- Attention, un `CommandController` n'est pas lié à un formulaire

Framework Spring MVC – Ancienne

➤ Vos formulaires :

- Hérite de SimpleFormController
- Gère les opérations liées à un formulaire
 - ☐ Population automatique des éléments saisis
 - ☐ Invocation d'une validation

➤ Ou peuvent faire usage d'un Wizard

- Hérite de AbstractWizardFormController
- Permet de gérer un formulaire saisi sur plusieurs pages
 - ☐ Un contrôleur peut être associé à plusieurs vues
 - ☐ Possibilité de paramétrer une méthode de validation par page

Framework Spring MVC – Ancienne

- Création de la classe contrôleur
- Configuration du contrôleur dans le contexte Spring
- Création de la classe Form associé au contrôleur si besoin
- Configuration du view resolver (mapping controller/jsp)
- Création de la page JSP et utilisation des tag lib Spring

Framework Spring MVC – Ancienne

➤ La Classe contrôleur

- Implémente l'interface Controller
- La méthode `handleRequest` renvoie un objet `ModelAndView`
 - ❑ Nom de la vue + données d'affichage

```
public class SimpleController implements Controller {  
  
    public ModelAndView handleRequest(HttpServletRequest request,  
        HttpServletResponse response) throws Exception {  
  
        Map map = new HashMap();  
        map.put("message", "ceci est un message a afficher");  
        map.put("id", "identification");  
        return new ModelAndView("jsp/main.jsp",map);  
    }  
}
```



En Spring 3+, il faut utiliser les annotations.

Framework Spring MVC – Ancienne

- Il vous faudra mapper chaque contrôleur dans le fichier Spring du DispatcherServlet

- Exemple de syntaxe pour le mapping

```
<bean id="handlerMapping"  
class="org.springframework.web.servlet.handler.BeanNameUrlHandlerMapping"/>  
  
<bean name="/simple.smvc" class="com.bankonet.SimpleController"/>
```



Contrairement aux utilisations précédentes, c'est l'attribut "**name**" qui est utilisé (et non-pas "id")

- Le suffixe n'est pas contractuel.
 - ❑ Il faut juste qu'il soit cohérent avec le suffixe dans le web.xml
- Il est possible d'ajouter des propriétés à ce bean
 - ❑ Ex : injection de dépendance avec une classe Service

Spring MVC et les annotations

- Spring 2.5 a introduit des annotations pour définir les contrôleurs
- Idées :
 - Supprimer la nécessité d'héritage
 - Privilégier le modèle POJO (annoter des classes POJO)
 - Alléger la configuration XML
 - ❑ Plus besoin du mapping

Framework Spring MVC – Moderne

➤ Le contrôleur (en annotations)

- Plus besoin d'héritage de classe/interface spécifique
- Plus besoin de surcharger une méthode spécifique

@Controller

@RequestMapping("/simple.smvc")

```
public class SimpleController {
```

@GetMapping

```
public String faireQQchose(HttpServletRequest request, HttpServletResponse response) throws Exception {
```

```
    Map map = new HashMap();
```

```
    map.put("message", "ceci est un message a afficher");
```

```
    map.put("id", "identification");
```

```
    request.setAttribute("map", map);
```

```
    return "jsp/main.jsp";
```

```
}
```

@PostMapping

```
public String faireAutreChose(HttpServletRequest request, HttpServletResponse response) throws Exception {
```

```
    ...
```

```
    return "jsp/main.jsp";
```

```
}
```

```
}
```

Le view resolver En fichier XML



- Cet élément a pour objectif de simplifier vos chemins d'URL dans le code
- La déclaration du ViewResolver se fait dans le fichier XML du Dispatcher :

```
<bean id="viewResolver" class="org.springframework.web.servlet.view.InternalResourceViewResolver">  
  <property name="viewClass" value="org.springframework.web.servlet.view.JstlView" />  
  <property name="prefix" value="/jsp/" />  
  <property name="suffix" value=".jsp/" />  
</bean>
```

- La propriété *viewClass* peut prendre d'autre valeur en fonction des framework (JSF par exemple) ou contraintes

Le view resolver Par classes Java



➤ Se fait dans la classe **WebMvcConfigurerAdapter**

```
@Configuration
@EnableWebMvc
@ComponentScan(basePackages = "com.controlleur")
public class MaConfigurationSpringMVC implements WebMvcConfigurer {

    @Bean
    public ViewResolver viewResolver() {
        InternalResourceViewResolver viewResolver = new InternalResourceViewResolver();
        viewResolver.setViewClass(JstlView.class);
        viewResolver.setPrefix("/jsp/");
        viewResolver.setSuffix(".jsp");
        return viewResolver;
    }
}
```

Le view resolver

➤ Exemple de transformation

➡ Dans le contrôleur :

```
Map map = new HashMap();  
map.put(...);  
return new ModelAndView("home",map);
```

➡ home ⇔ **/jsp/home.jsp**

 └───┬───┘ └───┬───┘
 prefix suffix

Le view resolver

➤ D'autres URLs possibles

- **redirect** ou **forward** : possibilité de prendre le chemin complet ou un nom court de vue

```
@PostMapping("/files/{path}")  
public String upload(...) {  
    // ...  
    return "redirect:files/{path}";  
}
```

```
@GetMapping("/")  
public String showMe(...) {  
    // ...  
    return "redirect:/login.html";  
}
```

Le view resolver

➤ Rappels

➤ Redirect :

- ❑ l'URL côté client change
- ❑ Un nouvel objet request est fabriqué

➤ Forward :

- ❑ l'URL côté client NE change PAS
- ❑ On passe les objets request et response au suivant

Le view resolver

➤ Vous pouvez aussi faire usage de ModelAndView

```
@PostMapping("/files/{path}")  
public ModelAndView upload(...) {  
    // ...  
    return new ModelAndView("redirect:files/{path}");  
}
```

```
@GetMapping("/")  
public ModelAndView showMe(...) {  
    // ...  
    return new ModelAndView("forward:/login.html");  
}
```

ModelAndView

➤ Contient :

- La vue (identifiant de la vue)
- Le modèle (ensemble des informations à afficher)

➤ Attention : la vue est le premier argument

➤ Spring-MVC place automatiquement la Map dans le contexte approprié (request par défaut)

```
Map map = new HashMap();  
map.put(...);  
return new ModelAndView("home",map);
```

Contrôleur exemple (1/3)

- Déclaration, instantiation et injection de dépendances
- Rappel : un contrôleur **n'est pas** un service métier

```
@Controller
@RequestMapping("/login.smvc")
public class LoginController {
    @Autowired
    private IClientService clientService;
    ...
}
```

Exemple :
Injection via
l'attribut

Contrôleur exemple (2/3)

➤ Affichage du formulaire Client

```
@Controller
@RequestMapping("/login.smvc")
public class LoginController {

    ...

    @GetMapping
    public String setupForm(ModelMap model) {
        model.addAttribute("client", new Client());
        return "clientForm";
    }
}
```

Contrôleur exemple (3/3)

➤ Soumission du formulaire Client

```
@Controller
@RequestMapping("/login.smvc")
public class LoginController {
    ...
    @PostMapping
    public String processSubmit(@ModelAttribute("client") Client client,
                               BindingResult error, HttpServletRequest request, ModelMap map) {

        client = clientService.findClientByNom(client.getNom());

        if(client == null) {
            error.reject(null, "Impossible de trouver le client !");
            return "clientForm";
        }

        map.addAttribute("client", client);
        return "clientDetail";
    }
}
```

Annotations Spring MVC, conclusion

- Les vues n'ont pas changé
- Approche 100% POJO
 - Plus à se préoccuper de quelle classe de contrôleur hériter
- Aucune configuration XML pour les contrôleurs
 - Vérification à la compilation
- Approche décentralisée
 - Avantage : pas de fichier de configuration à maintenir
 - Inconvénient : difficile d'avoir une vue globale
- Moins de réutilisabilité
- Nécessite des conventions de nommage pour des projets à grande échelle

Définition Spring MVC

➤ La signature des méthodes n'est pas figée.

Paramètres

Request, Response et Session de l'API Servlet	
@PathVariable	Récupère une partie de l'url et la place dans le paramètre annoté.
@RequestParam	Récupère un paramètre de requête web (GET ou POST)
@RequestHeader	Récupère un header de requête web
@RequestBody	Accède au corps de la requête
@ModelAttribute	Classe contenant le modèle objet et permet son enrichissement avant transmission à la vue

Retour

ModelAndView	Encapsule le nom logique de la vue et son modèle.
Model	Encapsule le modèle. Retour sur la vue courante.
View	Retourne la vue préparée par l'application. Exemple : retour de documents générés.
String	Fournit le nom logique

Gestion des vues

- Chaque méthode de contrôleur est chargée de sélectionner la vue consécutive à son traitement. Pour cela, elle renvoie
 - un nom logique en fin de traitement.

```
// Ou d'une autre manière si aucun modèle.
```

```
@RequestMapping("/helloWorld")  
public String helloWorld() {  
    return "helloWorld";  
}
```

- Ou un objet ModelAndView

```
@RequestMapping("/helloWorld2")  
public ModelAndView helloWorld() {  
    ModelAndView mav = new ModelAndView();  
    mav.setViewName("helloWorld");  
    mav.addObject("message", "Hello World!");  
    return mav;  
}
```

Gestion des vues

- Dans l'exemple précédent, la méthode `helloWorld` fournit à la vue les données par le biais du modèle transmis.
- La classe `ModelAndView` gère non seulement le nom logique de la vue mais aussi le modèle de la vue.
- Depuis la vue, l'attribut du modèle est directement référençable comme tout bean exposé à une page web.

Gestion du modèle

- Quand vous voulez faire passer un bean représentant le modèle, vous ferez usage de l'annotation `@ModelAttribute` sur un des paramètres de votre méthode
 - Le typage sera celui du bean qui vous intéresse

```
88+ * Soumet le formulaire.
98- @RequestMapping(value = "/dovirement.smvc", method = RequestMethod.POST)
99 public String doVirement(@ModelAttribute("virementBean") VirementBean virementBean, ModelMap modelMap,
100 BindingResult bindingResult) {
101     VirementController.LOG.debug("--> Passage dans doVirement");
```

Gestion du modèle

- **ModelAndView** : vue + modèle : on place dedans des informations en relation avec la vue.
 - Utiliser comme retour de méthode.
- **ModelMap** ou **Model** : On place dedans des informations (ce que vous voulez). C'est un peu le `xxxx.setAttribute` (ou `xxx` est un scope). Par défaut, tout va dans le scope request.
 - Utiliser comme paramètre de méthode.
- **@ModelAttribute** : permet de récupérer un bean qui sera automatiquement construit par Spring
 - Utiliser sur un paramètre de méthode ou une méthode pour cibler le code retour.
 - Fait le lien entre un formulaire web et le code

Gestion du modèle – scope session

- Pour rappel, vous pouvez ajouter un paramètre `HttpServletRequest` comme paramètre de vos méthodes
 - A partir de là, vous avez accès à tous les scopes
 - Mais c'est maladroit
- La gestion du scope des éléments que vous placez dans le Model / ModelAndView / ModelMap se fera via l'annotation **@SessionAttributes** (avec un 's')
 - C'est une annotation de classe
- Attention : l'annotation de classe **@Scope** n'a pas de rapport avec notre JEE mais avec le scope du contexte Spring !

Gestion du modèle – scope session

➤ Exemple

```
27 @Controller
28 @SessionAttributes({ "utilisateur" })
29 public class VirementController {
```

```
66+ * Affiche la page des virements.
72- @RequestMapping(value = "/virement.smvc", method = RequestMethod.GET)
73 public String showVirement(ModelMap modelMap) {
74     VirementController.LOG.debug("--> Passage dans showVirement");
75     VirementBean virementBean = new VirementBean();
76     modelMap.addAttribute("virementBean", virementBean);
77     // c'est l'annotation qui fait le lien avec la session
78     IUserisateurEntity utilisateur = (IUtilisateurEntity) modelMap.get("utilisateur");
79     if (utilisateur == null) {
80         VirementController.LOG.error("Erreur : utilisateur non connecte");
81         return "index";
82     }
83     this.getAndSetListeComptes(modelMap, utilisateur.getId());
84     return "comptes/virement";
85 }
```

Rappel : Gestion du scope Session

- En Spring 4.3+, plutôt que d'exposer l'objet HttpSession, pour une méthode, vous pouvez utiliser l'annotation de paramètre **@SessionAttribute** (sans 's')

```
@RequestMapping("/faire")  
public void helloWorld(@SessionAttribute("clef") User aUser) {  
    // ...  
}
```

Fait par le Spring

```
User aUser = (User) request.getSession(true).getAttribute("clef")
```

Rappel : Gestion du Scope Application

- Pour accéder au scope de l'application, vous devez ajouter un objet de type `javax.servlet.ServletContext`/`jakarta.servlet.ServletContext` dans votre contrôleur
- Et lui indiquer qu'il est en **@Autowired**
 - Ou, ne pas mettre **@Autowired** et implémenter l'interface `org.springframework.web.context.ServletContextAware`
- Votre attribut sera géré par le Spring et vous donnera accès au scope application

Rappel : Gestion du Scope Request

➤ Pour accéder à l'objet `HttpRequest`

- Vous pouvez simplement ajouter un attribut de type `HttpServletRequest` à vos méthodes

- ☐ Spring injectera la request automatiquement

- Ou, dans les cas qui ne permettent pas d'ajouter des paramètres, vous pouvez faire usage

- `RequestContextHolder.currentRequestAttributes()`

```
HttpServletRequest request = ((ServletRequestAttributes)  
RequestContextHolder.currentRequestAttributes()).getRequest();
```

Gestion des JSP

- Utilisation des tag-libs de Spring dans vos JSP
- Form :
- `<%@ taglib prefix="form" uri="http://www.springframework.org/tags/form" %>`
 - Permet de remplacer le formulaire et tous les *input* afin que Spring puisse les remplir/vider automatiquement

```
<form:form action="utilisateur.smv" method="post" commandName="utilisateurBean">

  <form:input path="nom" />
  <form:input path="prenom" />
  <input type="submit" value="Valider" />

</form:form>
```

- ❑ action = nom de l'action ⇔ @RequestMapping dans la classe Java
- ❑ commandName = nom d'un bean ⇔ clef d'un ModelMap associé dans votre classe Java
- ❑ path = référence une méthode du bean ⇔ utilisateurBean.getNom() / utilisateurBean.getPrenom() et utilisateurBean.setNom(valeur) / utilisateurBean.setPrenom(valeur)

Gestion des erreurs

- Pour la validation de vos formulaires vous pouvez faire usage des **Validator**
- De cette manière vos beans qui représentent vos formulaires restent simples
- C'est à vous de créer la classe et de la faire implémenter l'interface `org.springframework.validation.Validator`

Gestion des erreurs

- org.springframework.validation.**Validator** aura deux méthodes
 - **public boolean supports(Class<?> pArg0)**
 - ❑ Indique les éléments validables par cette classe
 - **public void validate(Object aLoginBean, Errors errors)**
 - ❑ Fait la validation
- Pour valider l'état d'un attribut, vous pouvez faire usage des méthodes statiques qui sont dans org.springframework.validation.**ValidationUtils**

Gestion des erreurs

- Vous ferez usage de vos Validator dans vos méthodes de contrôleurs
 - vous l'appellez sur votre bean en faisant usage du paramètre BindingResult qui se charge de porter les erreurs éventuelles
- **Note** : Le validator peut aussi être un attribut de votre contrôleur
 - Vous n'êtes pas obligé de l'instancier dans vos méthodes
 - Vous pouvez l'annoter en @Autowired (attention à ne pas oublier de poser un @Component sur votre classe de Validator)

Gestion des erreurs

```
* Soumet le formulaire.
@RequestMapping(value = "/dologin.smvc", method = RequestMethod.POST)
public String doLogin(@ModelAttribute("loginBean") LoginBean loginBean, ModelMap modelMap,
    BindingResult bindingResult) {
    LoginController.LOG.debug("--> Passage dans doLogin");

    LoginBeanValidator validator = new LoginBeanValidator();
    validator.validate(loginBean, bindingResult);

    if (!bindingResult.hasErrors()) {
        // Metier

        return "menu";
    }
    return "login";
}
```

Gestion des erreurs

- À tout moment dans vos méthodes de contrôleur, si vous avez l'objet **BindingResult** sous la main, vous pouvez ajouter des messages d'erreurs :

- Méthode reject ou addError

```
String msg = e.getMessage() != null ? e.getMessage() : e.getClass().getName();  
bindingResult.reject("error.technical", new Object[] { msg }, msg);  
HistoriqueController.LOG.error("Erreur:", e);
```

- ❑ reject : plus simple, moins prise de tête (plus global)
- ❑ addError : plus précis sur le contrôle (souvent pour un champ)

Gestion des erreurs

- Dans la page JSP vous pouvez récupérer vos erreurs via la balise *errors* :

```
<%@ taglib uri="http://www.springframework.org/tags/form" prefix="springForm"%>  
<springForm:errors path="login" cssClass="error" />
```

- S'il y a un message d'erreur associé à la clef login : alors, dans la page JSP, un bloc de `<span>` sera créé avec la classe *error*
- Vous pouvez indiquer « * » dans path pour lister toutes les erreurs
- Cette balise gère en automatique l'internationalisation
 - Il faut juste y penser en début de projet

Gestion des erreurs

- Vous pouvez aussi utiliser la balise *hasBindErrors* pour savoir s'il y a au moins une erreur sur le bean visé

```
<%@ taglib uri="http://www.springframework.org/tags" prefix="springTags"%>  
<springTags:hasBindErrors name="virementBean">...</springTags:hasBindErrors>
```

Gestion de l'internationalisation

- Pour que l'internationalisation fonctionne, vous devez déclarer un fichier properties sous l'id **messageSource**
- Cette déclaration se fait dans le fichier XML ou la classe (WebMvcConfigurerAdapter) de configuration DispatcherServlet

```
<bean id="messageSource" class="org.springframework.context.support.ReloadableResourceBundleMessageSource">  
  <property name="basename" value="classpath:/message"/>  
  <property name="defaultEncoding" value="utf-8"/>  
</bean>
```

```
@Bean  
public ReloadableResourceBundleMessageSource messageSource() {  
    ReloadableResourceBundleMessageSource messageSource = new ReloadableResourceBundleMessageSource();  
    messageSource.setBasename("classpath:/message");  
    messageSource.setDefaultEncoding("utf-8");  
    return messageSource;  
}
```

Gestion de l'internationalisation

- Dans l'exemple, le fichier properties portera comme nom :
 - message_[Locale].properties
 - Ex : message_fr_FR.properties pour le français de France
- Il contiendra toutes les clefs (code en Spring) pour les messages

```
9 error.wrong.password=Mauvais mot de passe
10 error.user.unknown=Utilisateur inconnu
11 error.technical=Erreur technique {0}
12 error.user.empty=Login vide
13 error.password.empty=Mot de passe vide
```

Gestion de l'internationalisation

- Dans votre JSP vous pouvez accéder à un message via la balise *message*

```
<%@ taglib uri="http://www.springframework.org/tags" prefix="springTags"%>  
<springTags:message code="votre.clef" />
```

- Vous pouvez aussi y inclure des éléments de JSTL
 - `<springTags:message code="${monbean.code}" />`

Les ressources statiques

- Les ressources statiques qui ne sont pas servies par un contrôleur devront être précisées
 - Typiquement quand le mapping URL du DispatcherServlet Spring se fait sur `"/**"`
- Cette déclaration se fait dans le fichier XML ou la classe (WebMvcConfigurerAdapter) de configuration DispatcherServlet

```
<mvc:annotation-driven />  
  
<mvc:resources mapping="/js/**" location="/js/" />  
<mvc:resources mapping="/css/**" location="/css/" />
```

```
@Override  
public void addResourceHandlers(ResourceHandlerRegistry pRegistry) {  
    pRegistry.addResourceHandler("/js/**").addResourceLocations("/js/");  
    pRegistry.addResourceHandler("/css/**").addResourceLocations("/css/");  
}
```

WebMvcConfigurerAdapter

Validation JSR 303

- En plus des **Validator** (ou à la place), vous pouvez faire usage de la norme Java JSR 303 pour valider vos beans
 - <http://beanvalidation.org/1.0/spec/>
- Objectifs de cette norme :
 - Proposer une spécification relative à la validation de données dans les applications Java
 - Fournir une API qui soit indépendante d'une architecture
 - Etre utilisable dans toutes les couches Java d'une application
 - ❑ Vous pouvez l'utiliser pour vos Bean, vos entités, ...
 - Standardiser la déclaration des contraintes en privilégiant **les annotations**
 - Etre facile à utiliser et extensible

Validation JSR 303 - Annotations

- Les annotations associées à cette norme peuvent s'appliquer sur :
 - La classe (ou interface)
 - ❑ Utilisées quand il faut manipuler plusieurs attributs de la classe en même temps afin de réaliser la validation
 - L'attribut
 - La méthode getXxx
- Toutes les annotations seront issues du package `javax.validation` (ou `jakarta.validation` si en JEE9+)

Validation JSR 303 - Annotations

Annotation	Description	Exemple
@AssertFalse	La valeur de l'attribut doit être false	<code>@AssertFalse</code> <code>boolean isUnsupported;</code>
@AssertTrue	La valeur de l'attribut doit être true	<code>@AssertTrue</code> <code>boolean isActive;</code>
@DecimalMax	La valeur doit être un chiffre à virgule inférieur ou égal à la valeur indiquée	<code>@DecimalMax("30.00")</code> <code>BigDecimal discount;</code>
@DecimalMin	La valeur doit être un chiffre à virgule supérieur ou égal à la valeur indiquée	<code>@DecimalMin("5.00")</code> <code>BigDecimal discount;</code>
@Digits	La valeur doit être un chiffre. Integer = nombre total de chiffres, fraction = nombre de chiffres après la virgule	<code>@Digits(integer=6, fraction=2)</code> <code>BigDecimal price;</code>
@Future	La valeur doit être une date dans le futur.	<code>@Future</code> <code>Date eventDate;</code>

Validation JSR 303 - Annotations

50

Annotation	Description	Exemple
@Max	La valeur doit être un chiffre entier inférieur ou égale à la valeur indiquée	@Max(10) int quantity;
@Min	La valeur doit être un chiffre entier supérieur ou égal à la valeur indiquée	@Min(5) int quantity;
@NotNull	La valeur ne doit pas être nulle	@NotNull String username;
@Null	La valeur doit être nulle	@Null String unusedString;
@Past	La valeur doit être une date dans le passé	@Past Date birthday;
@Pattern	La valeur doit respecter l'expression régulière.	@Pattern(regexp="^(\\d{3})\\d{3}-\\d{4}") String phoneNumber;
@Size	Fonctionne sur des valeurs de type String, Collection, Map, tableaux et permet de valider sa taille.	@Size(min=2, max=240) String briefMessage;

Validation JSR 303 - Spring

- Spring supporte la norme standard
- En Spring MVC elle s'utilisera avec l'annotation :
 - `@Valid` (ou `@Validated`) qui se place sur un paramètre de méthode d'un contrôleur
 - **Important** : l'attribut `BindingResult` **DOIT** être celui qui suit le bean à valider (sinon l'erreur sera un HTTP Status de 400)
- Les messages d'erreur, s'il y en a, iront dans le `BindingResult` automatiquement.
 - Le code du message sera :
 - ❑ `NomAnnotation.nomDeLobjetVise.nomAttribut`
 - Vous pouvez retrouver cette information en debuggant le contenu du `bindingresult`

Validation JSR 303 - Spring

- Si vous n'êtes pas en Spring Boot, pour que cela fonctionne, vous devrez aussi ajouter une dépendance vers un framework de validation

- Généralement celui d'hibernate

```
<!-- En Spring 5 -->
```

```
<dependency>
```

```
  <groupId>org.hibernate.validator</groupId>
```

```
  <artifactId>hibernate-validator</artifactId>
```

```
</dependency>
```

```
<!-- En Spring 6, il faudra ajouter en plus une dépendance vers -->
```

```
<dependency>
```

```
  <groupId>org.apache.tomcat.embed</groupId>
```

```
  <artifactId>tomcat-embed-el</artifactId>
```

```
</dependency>
```

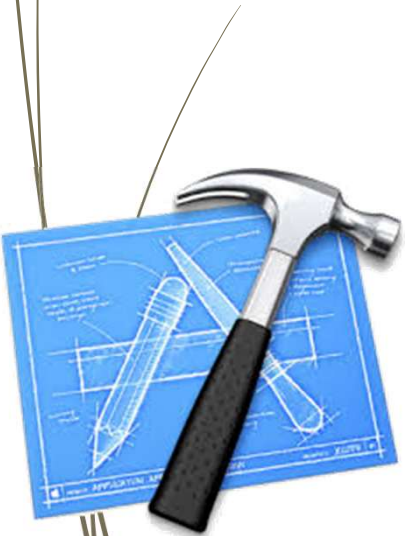
Validation JSR 303 - Spring

- Si vous êtes en Spring Boot 2.5+, pour que cela fonctionne, vous devrez aussi ajouter une dépendance vers un framework de validation

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-validation</artifactId>  
</dependency>
```

Travaux pratiques

- Exo 14 - Réaliser les travaux pratiques suivants:
 - Un projet web en Spring MVC + JPA



Template



- **Thymeleaf** : framework pour la génération de template WEB
- Un template peut être utilisé pour générer des pages dynamiquement
- C'est donc pour réaliser des pages Web Dynamiques qui **NE SONT PAS** des JSP
 - C'est une autre manière de faire des JSP

Template

- Dans le même esprit mais pas forcément ciblé Web vous avez les frameworks :
 - Freemarker (moteur de template du Struts)
 - Velocity (plus vraiment de première jeunesse)
- Freemarker ou Velocity peuvent vous permettre de faire des templates de texte (pas obligatoirement HTML)
 - Message pour un email par exemple.

Template avec Thymeleaf

- Thymeleaf est bien intégré à Spring et simplifie l'usage des templates
 - Il n'est pas recommandé de mélanger JSP et Thymeleaf l'un remplace l'autre
- Sa syntaxe est basée sur un name space XML :
 - `th:nomDeMaBalise`

Thymeleaf dépendances

➤ Dépendances côté Maven :

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
  <!-- Inclut le spring-boot-starter-web -->
</dependency>
```

En Spring
Boot

```
<dependency>
  <groupId>org.thymeleaf</groupId>
  <!-- Si vous êtes en Spring 3 -->
  <!-- <artifactId>thymeleaf-spring3</artifactId> -->
  <!-- En Spring 4 -->
  <artifactId>thymeleaf-spring4</artifactId>
  <!-- Si vous êtes en Spring 5 -->
  <!-- <artifactId>thymeleaf-spring5</artifactId> -->
  <version>3.0.11.RELEASE</version>
</dependency>
```

Sans Spring
Boot

Thymeleaf configuration Spring

➤ Côté configuration Spring - Spring boot

```
spring.thymeleaf.cache=true # Whether to enable template caching.
spring.thymeleaf.check-template=true # Whether to check that the template exists before rendering it.
spring.thymeleaf.check-template-location=true # Whether to check that the templates location exists.
spring.thymeleaf.enabled=true # Whether to enable Thymeleaf view resolution for Web frameworks.
spring.thymeleaf.enable-spring-el-compiler=false # Enable the SpringEL compiler in SpringEL expressions.
spring.thymeleaf.encoding=UTF-8 # Template files encoding.
spring.thymeleaf.excluded-view-names= # Comma-separated list of view names (patterns allowed) that should be excluded from resolution.
spring.thymeleaf.mode=HTML # Template mode to be applied to templates. See also Thymeleaf's TemplateMode enum.
spring.thymeleaf.prefix=classpath:/templates/ # Prefix that gets prepended to view names when building a URL.
# Comma-separated list of view names (patterns allowed) that should be the only ones executed in CHUNKED mode when a max chunk size is set.
spring.thymeleaf.reactive.chunked-mode-view-names=
# Comma-separated list of view names (patterns allowed) that should be executed in FULL mode even if a max chunk size is set.
spring.thymeleaf.reactive.full-mode-view-names=
spring.thymeleaf.reactive.max-chunk-size=0B # Maximum size of data buffers used for writing to the response.
spring.thymeleaf.reactive.media-types= # Media types supported by the view technology.
# Whether hidden form inputs acting as markers for checkboxes should be rendered before the checkbox element itself.
spring.thymeleaf.render-hidden-markers-before-checkboxes=false
spring.thymeleaf.servlet.content-type=text/html # Content-Type value written to HTTP responses.
# Whether Thymeleaf should start writing partial output as soon as possible or buffer until template processing is finished.
spring.thymeleaf.servlet.produce-partial-output-while-processing=true
spring.thymeleaf.suffix=.html # Suffix that gets appended to view names when building a URL.
spring.thymeleaf.template-resolver-order= # Order of the template resolver in the chain.
spring.thymeleaf.view-names= # Comma-separated list of view names (patterns allowed) that can be resolved.
```

Thymeleaf configuration Spring

➤ Côté configuration Spring - @Configuration

```
@Configuration
@EnableWebMvc
@ComponentScan(basePackages = "mes.controlleurs")
public class SpringConfigurationWeb implements WebMvcConfigurer {
    @Bean
    public ClassLoaderTemplateResolver templateResolver() {
        // ou un SpringResourceTemplateResolver
        ClassLoaderTemplateResolver templateResolver = new ClassLoaderTemplateResolver();
        templateResolver.setPrefix("templates/");
        templateResolver.setSuffix(".html");
        templateResolver.setTemplateMode(TemplateMode.HTML);
        templateResolver.setCharacterEncoding("UTF-8");
        templateResolver.setCacheable(true);
        templateResolver.setCheckExistence(true);
        return templateResolver;
    }
    @Bean
    public SpringTemplateEngine templateEngine() {
        SpringTemplateEngine templateEngine = new SpringTemplateEngine();
        templateEngine.setTemplateResolver(this.templateResolver());
        templateEngine.setEnableSpringELCompiler(true);
        return templateEngine;
    }
    ...
}
```

Où sont les
templates

Comment
gérer les
templates

Le View
Resolver du
Spring MVC

```
...
@Bean
public ThymeleafViewResolver viewResolver() {
    ThymeleafViewResolver viewResolver = new ThymeleafViewResolver();
    viewResolver.setTemplateEngine(this.templateEngine());
    viewResolver.setCharacterEncoding("UTF-8");
    return viewResolver;
}
...
```

Thymeleaf configuration Spring

➤ Côté configuration Spring - XML

```
<bean id="templateResolver" class="org.thymeleaf.templateresolver.ClassLoaderTemplateResolver">
  <property name="prefix" value="templates/" />
  <property name="suffix" value=".html" />
  <property name="templateMode" value="HTML" />
  <property name="cacheable" value="true" />
  <property name="characterEncoding" value="UTF-8" />
  <property name="checkExistence" value="true" />
</bean>

<bean id="templateEngine" class="org.thymeleaf.spring5.SpringTemplateEngine">
  <property name="templateResolver" ref="templateResolver" />
  <property name="enableSpringELCompiler" value="true" />
</bean>

<bean class="org.thymeleaf.spring5.view.ThymeleafViewResolver">
  <property name="templateEngine" ref="templateEngine" />
  <property name="characterEncoding" value="UTF-8" />
</bean>
```

Où sont les
templates

Comment
gérer les
templates

Le View
Resolver du
Spring MVC

springapp-servlet.xml

Thymeleaf – Structure côté front

- Les ressources web sont à placer dans le dossier
 - src/main/resources/static
 - Images / css / js / ...
- Les templates (pages web dynamiques) sont à placer dans le dossier
 - src/main/resources/templates
- Les fichiers internationalisés sont à placer dans
 - src/main/resources/xxx_fr_CA_.properties

Thymeleaf – Structure côté front

- Contrairement à une application JEE classique, il n'y a rien à placer dans webapp
 - Sauf, potentiellement
 - ❑ Dans meta-inf : fichiers de configuration pour la data-sources en déploiement JEE (sur serveur)
 - ❑ Dans WEB-INF : un fichier web.xml

Thymeleaf – Structure côté back

- Vous êtes en Spring classique
 - Vous aurez des @Repository / @Service

- Vous êtes en Spring MVC classique
 - @Controller
 - @GetMapping / @RequestMapping / ...
 - @SessionAttributes
 - @ModelAttribute
 - ModelMap & Model & BindingResult

Thymeleaf – Structure côté back

➤ Vos contrôleurs (en @Controller)

- Sont liés à des @Service par IOC
- Ont une URL (une route) principale via @RequestMapping
- Ont une/des méthodes en @XxxMapping pour indiquer des entry point
 - ❑ Elles retournent des chemins vers des TEMPLATE
 - return "login"; ⇔ fait référence au template /src/main/java/resources/templates/login.html
 - ❑ Prennent potentiellement en paramètre dans
 - @ModelAttribute : Pour récupérer de l'information de la vue précédente
 - ModelMap ou Model : pour pousser de l'information vers la vue suivante
 - BindingResult : pour gérer les erreurs

Thymeleaf – Syntaxe côté front

- Nous écrivons des pages HTML
- Elles peuvent / doivent faire usage des balises Thymeleaf
- Le namespace est, par défaut, "th"
- Dans nos balises HTML, nous ajouterons de th:xxx
 - `<link th:href="@{/css/bootstrap.min.css}" rel="stylesheet" />`

Thymeleaf – Syntaxe côté front

➤ Expressions :

- Expression avec des variables : `${...}`
- Expression vers une clef dans le fichier properties d'internationalisation : `#{...}`
- URL : `@{...}`

➤ Valeurs

- Text : 'one text', 'Another one!',...
- Chiffres : 0, 34, 3.0, 12.3,...
- Booléens : true, false
- null

Thymeleaf – Syntaxe côté front

- Opérateur sur les chaînes :

- +

- Opérations : +, -, *, /, %

- Boolean : and, or, !, not, >, <, >=, <= (gt, lt, ge, le), ==, != (eq, ne)

- Conditions

- If-then: (if) ? (then)

- If-then-else: (if) ? (then) : (else)

Thymeleaf – Syntaxe côté front

➤ Exemples :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="fr">
  <head>
    ...
    <title>Banque - Menu</title>
    <link th:href="@{/css/bootstrap.min.css}" rel="stylesheet" />
    ...
    <script th:src="@{/js/jquery.1.12.4.min.js}"></script>
    ...
  </head>
  <body class="container-fluid elbody" data-gatling="menu">
    

    <div class="alert alert-danger" th:if="${not #strings.isEmpty(erreur)}" th:utext="#{{erreur}}"></div>

    <div class="alert alert-info"
      th:text="#{welcome.message(${session.utilisateur.nom}, ${session.utilisateur.prenom})}"></div>
    ...
  </body>
</html>
```

Thymeleaf – Syntaxe côté front

➤ Exemples :

```
<form th:action="@{/dohistorique.smvc}" method="post">
  <div class="table-responsive"
    th:if="${not #lists.isEmpty('listeCompte')}}">
    <table class="table table-bordered table-striped" id="result">
      <caption>Liste de vos comptes sur Net Banque</caption>
      <thead class="bg-info">
        <tr>
          ...
        </tr>
      </thead>
      <tbody>
        <tr th:each="compte : ${listeCompte}"
          th:class="${compteStat.odd}? 'ellignetableau1':'ellignetableau2'">

          <td class="ellibelletableau text-center"
            th:attr="data-cpid=${compte.id}">
            <a th:href="javascript:go('${compte.id}+')"
              th:utext="${compte.id}"></a>
          </td>
          <td class="ellibelletableau" th:utext="${compte.libelle}"></td>
        </tr>
      </tbody>
    </table>
  </div>
</form>
```

Thymeleaf – Syntaxe côté front

➤ Couplage avec Spring MVC

➤ Formulaire :

```
<form th:action="@{/dologin.smvc}" method="post" th:object="${loginBean}" class="form-horizontal">  
<form:form action="dologin.smvc" method="post" modelAttribute="loginBean" class="form-horizontal">
```

➤ Champs :

```
<input type="text" th:field="*{login}" class="form-control" placeholder="Login" id="inLogin" />  
<form:input path="login" class="form-control" placeholder="Login" id="inLogin" />
```

➤ Gestion des erreurs :

```
<div class="alert alert-danger" th:if="${#fields.hasErrors('*')}" th:errors="*"></div>  
<spring:hasBindErrors name="loginBean"><div class="alert alert-danger"><form:errors path="*" /></div></spring:hasBindErrors>
```

Travaux pratiques

- Exo 26 - Réaliser les travaux pratiques suivants:
 - Un projet web en Spring MVC + JPA + Thymeleaf

